Università degli studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Analisi di sicurezza e integrità crittografica di un sistema di versionamento distribuito

Tesi di laurea triennale

Relatore

Prof. Vardanega

Laureando

Michele Stevanin

Matricola 2101741

ANNO ACCADEMICO 2025-2026

© Michele Stevanin, Luglio 2026. Tutti i diritti riservati. Tesi di laurea triennale: *Analisi di sicurezza e integrità crittografica di un sistema di versionamento distribuito.*

«Chi non conosce la storia è condannato a ripeterla,
ma chi la conosce è condannato a guardare gli altri che la ripetono.»

— Primo Levi

Sommario

Questa relazione descrive il lavoro svolto durante lo stage curricolare presso Zucchetti S.p.A., della durata di trecentoquattro ore, sotto la supervisione del tutor aziendale Gregorio Piccoli e del tutor accademico Prof. Vardanega.

Lo stage ha riguardato l'analisi della sicurezza di *RVC*$_G$, un sistema di versionamento distribuito sviluppato internamente da Zucchetti S.p.A. come alternativa a *Git*$_G$. A differenza dei sistemi tradizionali, *RVC*$_G$ non richiede un server centrale per la verifica di autenticità dei *commit*$_G$: i *commit*$_G$ vengono distribuiti come archivi firmati, navigabili direttamente tramite filesystem. L'integrità dei contenuti è garantita attraverso la verifica crittografica degli *hash*$_G$ di ogni *commit*$_G$, con l'obiettivo di permettere a qualsiasi utente di accertare autonomamente l'autenticità della *repository*$_G$ ricevuta, indipendentemente dalla fonte.

Il lavoro ha previsto lo studio delle tecnologie crittografiche alla base del sistema — chiavi *SSH*$_G$, firma-digitale e strumenti di cifratura — seguito dalla simulazione di scenari di attacco con diversi livelli di accesso. Per ciascuno scenario sono state individuate le vulnerabilità presenti, analizzate le possibili conseguenze e proposte contromisure concrete.

Organizzazione del testo

Il primo capitolo presenta l'azienda ospitante, introduce il progetto *RVC*$_G$ e illustra le motivazioni che mi hanno portato a scegliere questo stage, con gli obiettivi definiti, la pianificazione e l'analisi dei rischi;

Il secondo capitolo descrive l'organizzazione del lavoro durante il tirocinio, l'ambiente di sviluppo, gli strumenti utilizzati e l'approccio metodologico seguito;

Il terzo capitolo illustra le tecnologie e i concetti teorici alla base del progetto, dalla crittografia-asimmetrica e *SSH*$_G$ fino all'architettura di *RVC*$_G$ e al confronto con *Git*$_G$

Il quarto capitolo espone l'analisi del codice sorgente *CPL*$_G$ di *RVC*$_G$, descrive le vulnerabilità individuate e presenta il modello di minaccia costruito secondo il framework Mitre-attack;

Il quinto capitolo presenta gli scenari di attacco simulati in ambiente controllato, con diversi livelli di accesso — da attaccante esterno fino a compromissione della chiave del capo progetto — e analizza la propagazione degli errori nella catena degli *hash*$_G$

Il sesto capitolo descrive i miglioramenti implementati nel codice sorgente di *RVC*$_G$, tra cui il sistema di configurazione dinamico, la firma *SSH*$_G$ integrata, la verifica dell'integrità della catena e la gestione degli *allowed signers*;

Il settimo capitolo riassume i risultati raggiunti, valuta il grado di soddisfacimento degli obiettivi prefissati e propone una riflessione personale sull'esperienza e sugli sviluppi futuri.

Convenzioni tipografiche

Durante la stesura del testo sono state adottate le seguenti convenzioni tipografiche:

- Gli acronimi, le abbreviazioni e i termini di uso non comune vengono definiti nel glossario, situato alla fine del documento (p. 33);
- I termini presenti nel glossario sono indicati con la notazione: *RVC*$_G$;
- I termini in lingua straniera non di uso comune o appartenenti al gergo tecnico sono evidenziati in *corsivo*;
- I nomi di funzioni, variabili o comandi sono scritti con carattere `monospaziato`;
- I riferimenti bibliografici sono indicati con il numero identificativo della fonte, es. [1];
- I blocchi di codice sorgente sono rappresentati nel seguente modo:

```cpl
proc ReportInfo(FileManifestPersistent fm, bool files:=false)
  if fm.tag <> nil
    ? '    tag:', fm.tag
  end
  if fm.prev <> nil
    ? '   prev:', fm.prev
  end
  if fm.merge <> nil
    ? '  merge:', fm.merge
  end
end
```

Esempio di funzione CPL estratta dal sorgente di *RVC*$_G$.

Indice

1 Introduzione ..... **1.**

1.1 L'azienda ..... 1.
1.2 Il progetto ..... 2.
1.3 Scelta del progetto ..... 2.
1.4 Obiettivi dello stage ..... 3.
1.5 Pianificazione ..... 4.
1.6 Analisi dei rischi ..... 5.

2 Descrizione dello stage ..... **7.**

2.1 Organizzazione del lavoro ..... 7.
2.1.1 Rapporto con i tutor ..... 7.
2.2 Ambiente di sviluppo ..... 8.
2.2.1 Strumenti utilizzati ..... 8.
2.3 Approccio metodologico ..... 9.
2.3.1 Ambiente di simulazione ..... 9.

3 Tecnologie e fondamenti teorici ..... **11.**

3.1 Crittografia asimmetrica e firma digitale ..... 11.
3.1.1 La firma digitale in dettaglio ..... 12.
3.1.2 Curve ellittiche: Ed25519 ..... 13.
3.2 SSH e ssh-keygen ..... 13.
3.2.1 Autenticazione con chiavi ..... 13.
3.2.2 ssh-keygen per la firma di file ..... 14.
3.3 AGE ..... 14.
3.4 Sistemi di controllo versione ..... 15.
3.4.1 Storia e evoluzione ..... 15.
3.4.1.1 Sistemi centralizzati ..... 15.
3.4.1.2 Sistemi distribuiti ..... 16.

3.4.2 Git in dettaglio ..... 17.
3.4.3 RVC a confronto con Git ..... 17.
3.5 RVC: architettura e funzionamento ..... 18.
3.5.1 Struttura della repository ..... 18.
3.5.2 Il file .sig e la blockchain degli hash ..... 18.
3.5.3 Il linguaggio CPL ..... 19.
3.5.4 Flusso di un commit ..... 20.

4 Requisiti e modello di sicurezza ..... 21.

4.1 Sicurezza strutturale nei sistemi di versionamento distribuito ..... 21.
4.1.1 Confronto con i sistemi esistenti ..... 22.
4.1.2 Applicazione in RVC ..... 23.
4.2 Requisiti di sicurezza formali ..... 23.
4.3 Gap analysis ..... 25.

5 Simulazione scenari di attacco ..... 27.

6 Miglioramenti implementati ..... 29.

7 Conclusioni ..... 31.

7.1 Consuntivo finale ..... 31.
7.2 Raggiungimento degli obiettivi ..... 31.
7.3 Requisiti soddisfatti ..... 31.
7.4 Rischi occorsi e mitigati ..... 32.
7.5 Valutazione personale ..... 32.

Glossario ..... 33.

Elenco delle Figure

1.1 Logo di Zucchetti S.p.A. . . . . 1.
3.1 Flusso trasferimento messaggio con uso di chiave pubblica e privata . . . . . 12.
3.2 Flusso firma di un documento con chiave pubblica e privata . . . . . 12.
3.3 Comparazione dimensione tra tipologie di chiavi . . . . . 13.
3.4 Esempio di un file cifrato con AGE . . . . . 15.
3.5 Controllo versioni centralizzato vs distribuito . . . . . 16.
3.6 Struttura del file .sig e la catena degli hash cumulativi . . . . . 19.
3.7 Struttura dei moduli principali di RVC . . . . . 20.

Elenco delle Tabelle

1.1 Obiettivi dello stage. . . . . 3.
1.2 Pianificazione del periodo di stage. . . . . 4.
1.3 Analisi preventiva dei rischi. . . . . 5.
3.1 Confronto tra Git e RVC. . . . . 17.
4.1 Requisiti di sicurezza del modello proposto. . . . . 24.
7.1 Consuntivo orario finale. . . . . 31.
7.2 Riepilogo dei requisiti soddisfatti. . . . . 32.
7.3 Rischi occorsi con la loro mitigazione. . . . . 32.

Elenco dei Codici Sorgente

0.1 Esempio di funzione CPL estratta dal sorgente di RVC$_G$. vi

Capitolo 1

Introduzione

In questo capitolo viene descritta l'azienda ospitante, introdotto il progetto e illustrate le motivazioni che hanno portato alla scelta di questo stage.

1.1 L'azienda

Zucchetti S.p.A. è un'azienda italiana con sede principale a Lodi che produce soluzioni software, hardware e servizi per aziende, professionisti e associazioni di categoria. Le origini risalgono al 1977, quando lo studio del commercialista Domenico «Mino» Zucchetti realizzò il primo software in Italia per l'elaborazione automatica delle dichiarazioni dei redditi. L'anno successivo, nel 1978, fu fondata formalmente l'azienda.

Oggi il gruppo conta più di 8.000 dipendenti, di cui 2.000 dedicati alla Ricerca e Sviluppo, e serve oltre 700.000 clienti tramite sedi distribuite in tutta Italia e più di 1.650 partner. A livello internazionale è presente con proprie società in diversi paesi europei e oltreoceano, con una rete di oltre 350 partner in 50 paesi. Zucchetti si posiziona oggi come la prima *software house* in Italia per fatturato.

Lo stage è stato svolto presso la sede di Padova e di Noventa Padovana (PD), sotto la supervisione del tutor aziendale Gregorio Piccoli.

Logo di Zucchetti S.p.A.

1.2 Il progetto

Il progetto ha riguardato l'analisi della sicurezza di *RVC*$_G$, un sistema di versionamento distribuito sviluppato internamente da Zucchetti S.p.A. come alternativa a *Git*$_G$. A differenza dei sistemi tradizionali, *RVC*$_G$ non richiede un server centrale: i *commit* vengono distribuiti come archivi firmati, navigabili direttamente tramite filesystem. L'integrità dei contenuti è garantita attraverso la verifica crittografica degli *hash*$_G$ di ogni *commit*, con l'obiettivo di permettere a qualsiasi utente di accertare autonomamente l'autenticità della *repository* ricevuta, indipendentemente dalla fonte di distribuzione.

L'autenticazione e la firma dei *commit* avvengono tramite chiavi *SSH*$_G$, rendendo ogni modifica crittograficamente attribuibile al suo autore. Questo approccio distingue *RVC*$_G$ non solo per l'architettura distribuita, ma anche per le garanzie di autenticità che offre rispetto ai sistemi di versionamento convenzionali.

La versione di *RVC*$_G$ fornita per lo stage è una versione di sviluppo deliberatamente priva di alcuni meccanismi di sicurezza, con l'obiettivo di permettere uno studio autonomo delle vulnerabilità e la progettazione di soluzioni originali. Questo approccio ha consentito di affrontare il problema della sicurezza senza vincoli architetturali predefiniti, producendo analisi e implementazioni indipendenti.

1.3 Scelta del progetto

Ho scelto questo progetto per l'interesse verso la sicurezza informatica e la crittografia applicata, temi che avevo già incontrato durante il percorso universitario ma che non avevo mai avuto l'occasione di approfondire in un contesto reale. L'analisi di un sistema in uso aziendale, con l'obiettivo di individuare vulnerabilità concrete e proporre miglioramenti, rappresenta un'opportunità difficilmente replicabile in ambito accademico.

La natura del lavoro — che combina studio teorico delle tecnologie crittografiche e simulazione pratica di scenari di attacco — mi ha convinto che fosse il progetto più adatto per concludere il percorso triennale con un contributo tangibile.

1.4 Obiettivi dello stage

Gli obiettivi dello stage sono stati definiti in accordo con il tutor aziendale e classificati secondo tre livelli di priorità:

- **Obbligatori** (O): obiettivi il cui soddisfacimento è vincolante per la riuscita del progetto;
- **Desiderabili** (D): obiettivi non strettamente necessari ma dal riconoscibile valore aggiunto;
- **Facoltativi** (F): obiettivi che rappresentano un ulteriore contributo non competitivo.

Codice	Descrizione
O01	Studio delle tecnologie *SSH*$_G$, *AGE*$_G$ e *RVC*$_G$
O02	Analisi del sistema *RVC*$_G$ e individuazione delle vulnerabilità
O03	Simulazione di attacchi senza possesso di credenziali
O04	Simulazione di attacchi con chiave-privata compromessa
O05	Produzione della documentazione tecnica e della relazione finale
D01	Simulazione di attacchi con chiave del capo progetto compromessa
D02	Gestione dei *signers* per *branch* diversi all'interno della *repository*
D03	Studio e implementazione delle tecniche di recovery delle credenziali
F01	Analisi delle possibilità di adozione di una struttura monorepo o polirepo in *RVC*$_G$
F02	Progettazione e sviluppo di un sistema di *repository* cifrato con distribuzione dei permessi

Obiettivi dello stage.

1.5 Pianificazione

Il lavoro è stato organizzato su otto settimane per un totale di 304 ore, suddivise tra studio delle tecnologie, analisi della sicurezza, sviluppo e documentazione.

Settimana	Ore	Attività
1	32	Studio di *SSH*$_G$, crittografia-asimmetrica, firma-digitale e *AGE*$_G$
2	40	Studio di *RVC*$_G$: architettura, formato dei *commit*$_G$
3	40	Analisi del codice sorgente *CPL*$_G$ e individuazione delle vulnerabilità
4	40	Simulazione scenari di attacco senza credenziali e con chiave compromessa
5	40	Implementazione miglioramenti: configurazione, firma *SSH*$_G$, verifica integrità
6	32	Gestione signers, allowed_signers, verifica catena completa
7	40	Simulazione attacchi avanzati e sviluppi futuri
8	40	Completamento e revisione della relazione finale

Pianificazione del periodo di stage.

1.6 Analisi dei rischi

Prima dell'avvio del progetto è stata condotta un'analisi preventiva dei rischi, al fine di individuare le possibili criticità e predisporre le opportune contromisure.

Descrizione	**Contromisura**	**Probabilità** **Impatto**
Codice sorgente di *RVC*$_G$ non disponibile nella fase iniziale, con impossibilità di analisi *white-box*$_G$	Analisi *black-box*$_G$ tramite osservazione del comportamento esterno e dei file prodotti	Alta Alto
Linguaggio *CPL*$_G$ proprietario senza documentazione pubblica, con curva di apprendimento elevata	Studio della documentazione interna fornita dall'azienda e confronto diretto col tutor	Alta Medio
Comportamenti silenziosi del sistema in caso di errore, che rendono difficile il debug	Aggiunta sistematica di istruzioni di debug nel codice sorgente durante l'analisi	Media Alto
Vulnerabilità individuate già risolte nella versione interna, rendendo il lavoro ridondante	Verifica periodica col tutor aziendale sull'allineamento tra la versione di test e quella interna	Media Medio

Analisi preventiva dei rischi.

Capitolo 2

Descrizione dello stage

In questo capitolo viene descritta l'organizzazione del lavoro durante il tirocinio, l'ambiente di sviluppo utilizzato, gli strumenti adottati e l'approccio metodologico seguito.

2.1 Organizzazione del lavoro

Lo stage si è svolto in presenza presso le sedi di Zucchetti S.p.A., con rare eccezioni in modalità *smart working* nei periodi in cui l'azienda era impegnata in un trasferimento di sede. La prima settimana si è tenuta presso gli uffici di via Giovanni Cittadella a Padova; le settimane successive presso la sede principale di Noventa Padovana (PD), dove ho trascorso la maggior parte del tirocinio.

L'orario di lavoro era strutturato su due fasce giornaliere: dalle 9:00 alle 13:00 e dalle 14:00 alle 18:00, per un totale di otto ore al giorno. Questa organizzazione ha permesso di alternare sessioni di studio teorico al mattino con attività pratiche nel pomeriggio, sfruttando la pausa di mezzogiorno per consolidare i concetti appresi.

2.1.1 Rapporto con i tutor

Il confronto con i tutor ha avuto modalità diverse a seconda del ruolo.

Il **tutor aziendale**, Gregorio Piccoli, era disponibile quasi quotidianamente per chiarimenti, revisioni del lavoro svolto e indicazioni sui passi successivi. Le riunioni non seguivano una cadenza fissa ma avvenivano su richiesta, ogni volta che si presentava un problema tecnico rilevante o si raggiungeva un risultato degno di discussione. Questo approccio ha favorito un dialogo continuo e ha permesso di adattare il percorso in modo flessibile all'avanzamento del lavoro.

Il **tutor accademico**, il Professor Prof. Vardanega, è stato invece coinvolto con cadenza settimanale tramite scambio di email. Le comunicazioni riguardavano principalmente l'avanzamento dello stage rispetto alle attese previste e la verifica dell'allineamento con gli obiettivi didattici dello stage.

2.2 Ambiente di sviluppo

Per tutta la durata dello stage ho utilizzato un portatile fornito dall'azienda con sistema operativo Windows. La scelta degli strumenti da installare è stata lasciata alla mia discrezione, senza vincoli imposti dall'azienda, il che ha permesso di configurare un ambiente di lavoro ottimale per le esigenze del progetto.

L'accesso ai sistemi aziendali includeva anche i modelli linguistici (*LLM*) disponibili localmente nell'infrastruttura di Zucchetti, utilizzati come supporto durante le fasi di studio e sviluppo.

2.2.1 Strumenti utilizzati

Gli strumenti principali adottati durante lo stage sono stati:

Visual Studio Code come editor di testo principale, utilizzato sia per la scrittura e modifica del codice sorgente *CPL*$_G$ di *RVC*$_G$, sia per la stesura della documentazione in Typst. L'editor è stato configurato con estensioni per la sintassi *CPL*$_G$ e per la compilazione live dei documenti Typst.

GitHub per il versionamento della tesi e dei file di lavoro personali. Il *repository*$_G$ della relazione finale è ospitato su GitHub con pubblicazione automatica del PDF tramite *GitHub Actions* e *GitHub Pages*.

Typst come sistema di composizione tipografica per la stesura della relazione finale. Typst è un'alternativa moderna a LaTeX che permette di produrre documenti PDF di qualità professionale con una sintassi più accessibile.

Terminale Windows (cmd e PowerShell) per l'esecuzione dei comandi *RVC*$_G$, la gestione dei file e le operazioni di debug. La maggior parte delle operazioni con *RVC*$_G$ avviene tramite interfaccia a riga di comando.

PuTTY come client *SSH*$_G$ per Windows, utilizzato per la gestione delle chiavi *SSH*$_G$ tramite il componente `puttygen`.

Ssh-keygen (OpenSSH per Windows) per la generazione delle chiavi *Ed25519*$_G$, la firma crittografica dei file e la verifica delle firme *SSH*$_G$. Questo strumento è centrale nel meccanismo di sicurezza implementato in *RVC*$_G$.

AGE$_G$ come strumento di cifratura moderno, studiato durante lo stage in preparazione alla fase di progettazione delle *repository* cifrate prevista negli obiettivi desiderabili.

2.3 Approccio metodologico

Il percorso di lavoro ha seguito un andamento alternato tra fasi teoriche e fasi pratiche, con transizioni determinate dall'avanzamento della comprensione del sistema e dalla disponibilità del materiale.

Nella **fase iniziale** l'attenzione era rivolta allo studio delle tecnologie crittografiche — chiavi *SSH*$_G$, firma-digitale, *AGE*$_G$ — attraverso documentazione ufficiale, esempi pratici al terminale e confronto con il tutor aziendale. Parallelamente ho iniziato a esplorare *RVC*$_G$ dall'esterno, analizzandone il comportamento tramite i comandi disponibili e studiando il formato dei file prodotti.

Una volta ottenuto accesso ai **sorgenti *CPL*$_G$**, l'approccio è cambiato: dall'analisi esterna (*black-box*) si è passati all'analisi interna (*white-box*), con lettura sistematica del codice, identificazione delle vulnerabilità e progettazione degli interventi migliorativi. Questa fase ha richiesto anche lo studio del linguaggio *CPL*$_G$, per il quale non esiste documentazione pubblica — la comprensione è avvenuta tramite lettura del codice esistente e consultazione della documentazione interna fornita dall'azienda.

La **fase di implementazione** ha proceduto in parallelo con l'analisi: man mano che venivano individuate vulnerabilità o carenze, venivano progettate e implementate le relative soluzioni, testandole su una *repository* di simulazione appositamente creata.

2.3.1 Ambiente di simulazione

Per riprodurre scenari di attacco in modo controllato e reversibile, ho configurato un ambiente di test dedicato composto da:

- Una **directory di lavoro** (`simulazione/`) contenente i file di un progetto fittizio su cui eseguire le operazioni *RVC*$_G$.
- Una ***repository*$_G$ locale** (`repo/`) dove vengono archiviati i *commit*, separata dalla directory di lavoro.
- Un file `.git/repository.info` che collega la directory di lavoro alla *repository*$_G$, seguendo la convenzione di *RVC*$_G$.
- Un file `allowed_signers` contenente le chiavi pubbliche degli autori autorizzati a firmare i *commit*, utilizzato per i test di verifica dell'integrità.

Questo ambiente ha permesso di simulare scenari realistici — inclusi la manomissione dei file di *commit*, la compromissione delle chiavi *SSH*$_G$ e la verifica della propagazione degli errori nella catena degli *hash*$_G$ — senza rischiare di danneggiare dati reali.

Capitolo 3

Tecnologie e fondamenti teorici

In questo capitolo sono illustrate le tecnologie e i concetti teorici alla base del progetto. Partendo dalla crittografia-asimmetrica, vengono descritti SSH$_G$, AGE$_G$ e i sistemi di controllo versione, fino ad arrivare all'architettura di RVC$_G$.

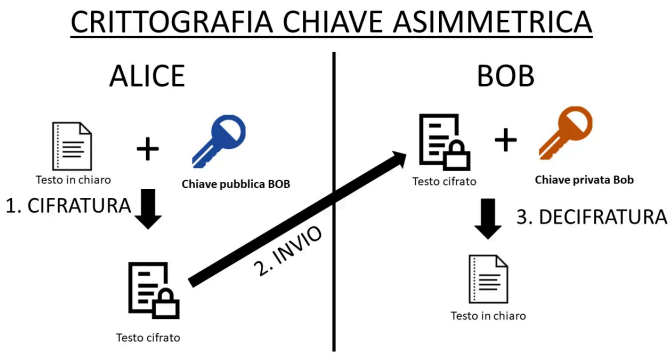
3.1 Crittografia asimmetrica e firma digitale

La crittografia è la disciplina che studia le tecniche per proteggere le informazioni rendendole illeggibili a chi non è autorizzato. Nella sua forma classica, detta *crittografia simmetrica*, mittente e destinatario condividono una stessa chiave segreta per cifrare e decifrare i messaggi. Questo approccio presenta un problema fondamentale: come si trasmette la chiave in modo sicuro prima ancora di poter comunicare in modo sicuro?

La risposta a questo problema arrivò negli anni "70 con la *crittografia asimmetrica*, detta anche *crittografia a chiave pubblica*. L"idea è elegante: ogni utente genera una coppia di chiavi matematicamente collegate tra loro. La **chiave-pubblica** può essere distribuita liberamente a chiunque. La **chiave-privata** deve rimanere segreta e non lasciare mai il dispositivo del proprietario. Le due chiavi sono collegate da una relazione matematica tale per cui ciò che viene cifrato con una può essere decifrato solo con l'altra, ma è computazionalmente impossibile ricavare la chiave-privata a partire da quella pubblica.

Questo meccanismo permette due operazioni fondamentali:

- **Cifratura**: chiunque può cifrare un messaggio usando la chiave-pubblica del destinatario. Solo il destinatario, possedendo la chiave-privata corrispondente, potrà decifrarlo.
- **Firma-digitale**: il mittente cifra un messaggio con la propria chiave-privata. Chiunque, usando la chiave-pubblica del mittente, può verificare che il messaggio provenga effettivamente da lui e non sia stato alterato.



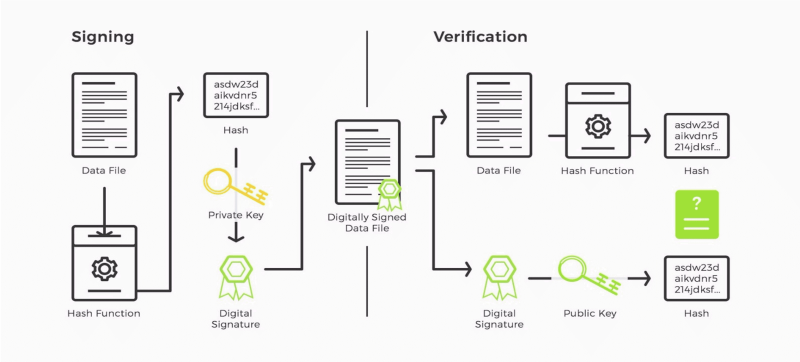
Flusso trasferimento messaggio con uso di chiave pubblica e privata

3.1.1 La firma digitale in dettaglio

La firma-digitale non cifra il messaggio intero — sarebbe inefficiente. Funziona invece in questo modo:

1. Il mittente calcola l'*hash* del messaggio, ovvero una sua impronta digitale di dimensione fissa prodotta da una funzione matematica non invertibile.
2. Il mittente cifra quell'hash con la propria chiave-privata. Il risultato è la firma-digitale.
3. Il destinatario riceve messaggio e firma. Decifra la firma usando la chiave-pubblica del mittente, ottenendo l'hash originale.
4. Il destinatario calcola autonomamente l'hash del messaggio ricevuto e confronta i due valori. Se coincidono, la firma è valida: il messaggio proviene dal mittente dichiarato e non è stato modificato.

La sicurezza di questo meccanismo si basa su due proprietà: la resistenza alle collisioni delle funzioni di $hash_G$ (è praticamente impossibile trovare due messaggi diversi con lo stesso $hash_G$) e l'impossibilità computazionale di produrre una firma valida senza possedere la chiave-privata.



Flusso firma di un documento con chiave pubblica e privata

3.1.2 Curve ellittiche: Ed25519

Gli algoritmi crittografici moderni si basano su problemi matematici computazionalmente difficili. RSA, il primo algoritmo a chiave-pubblica diffuso, si basa sulla difficoltà di fattorizzare numeri molto grandi. Gli algoritmi moderni basati su *curve ellittiche* offrono lo stesso livello di sicurezza con chiavi molto più corte, risultando più veloci ed efficienti.

Ed25519$_G$ è un algoritmo di firma-digitale basato sulla curva ellittica Curve25519. Produce chiavi di 256 bit e firme di 512 bit, con un livello di sicurezza equivalente a RSA con chiavi da 3000 bit. È l'algoritmo raccomandato oggi per la firma *SSH*$_G$ ed è quello utilizzato in questo progetto.

Symmetric Key Size (bits)	RSA and Diffie-Hellman Key Size (bits)	Elliptic Curve Key Size (bits)
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	521

Comparazione dimensione tra tipologie di chiavi

3.2 SSH e ssh-keygen

SSH$_G$ è un protocollo di rete che permette di stabilire connessioni sicure tra due macchine attraverso una rete non sicura. Nato come sostituto sicuro di Telnet e rsh, *SSH*$_G$ cifra tutto il traffico e autentica sia il server che il client, impedendo attacchi di tipo *man-in-the-middle*.

3.2.1 Autenticazione con chiavi

SSH$_G$ supporta due modalità principali di autenticazione: tramite password e tramite coppia di chiavi. L'autenticazione con chiavi è considerata più sicura e viene raccomandata in tutti i contesti professionali. Il funzionamento è il seguente:

1. L'utente genera una coppia di chiavi con `ssh-keygen`.
2. La chiave-pubblica viene copiata sul server remoto, nella cartella `~/.ssh/authorized_keys`.
3. Al momento della connessione, il server invia una sfida cifrata con la chiave-pubblica dell'utente.
4. Il client dimostra di possedere la chiave-privata corrispondente risolvendo la sfida.
5. La connessione viene stabilita senza che la chiave-privata abbia mai lasciato il client.

3.2.2 ssh-keygen per la firma di file

Oltre alla gestione delle chiavi *SSH*$_G$, `ssh-keygen` offre una funzionalità meno nota ma molto utile: la firma e verifica di file arbitrari. Questo è il meccanismo che *RVC*$_G$ utilizza per firmare crittograficamente i *commit*.

Il comando per firmare un file è:

```bash
ssh-keygen -Y sign -n <namespace> -f <chiave_privata> <file>
```

Il parametro `-n` specifica il *namespace*, una stringa che identifica il contesto d'uso della firma e previene attacchi di riutilizzo — una firma prodotta con *namespace*$_G$ `git` non può essere spacciata per una firma con *namespace*$_G$ `file`.

Il comando per verificare una firma è:

```bash
ssh-keygen -Y verify -n <namespace> -f <allowed_signers> -I <identità> -s <firma> < <file>
```

Il file `allowed_signers` è un elenco di identità autorizzate con le rispettive chiavi pubbliche, nel formato:

```
identita@esempio.com ssh-ed25519 AAAA...chiave...
```

Se la firma è valida e l'identità è presente nel file, il comando restituisce `Good "file" signature for <identità>`.

3.3 AGE

AGE$_G$ (*Actually Good Encryption*) è uno strumento moderno per la cifratura di file, progettato con l'obiettivo di essere semplice, sicuro e componibile. A differenza di PGP, che nel corso degli anni ha accumulato una complessità notevole, *AGE*$_G$ offre un'interfaccia minimale con poche opzioni ben definite.

AGE$_G$ supporta tre modalità di cifratura:

- **Chiave-pubblica**: il file viene cifrato con la chiave-pubblica del destinatario e può essere decifrato solo con la corrispondente chiave-privata.
- **Chiave *SSH***$_G$: *AGE*$_G$ può utilizzare le stesse chiavi *SSH*$_G$ già esistenti (incluse le chiavi *Ed25519*$_G$) come chiavi di cifratura, senza bisogno di generare nuove chiavi dedicate.
- ***Passphrase***$_G$: il file viene cifrato con una *passphrase*$_G$, usando la funzione di derivazione `scrypt` per proteggersi da attacchi a dizionario.

Un file cifrato con *AGE*$_G$ contiene nell'intestazione le informazioni necessarie per la decifratura (il tipo di chiave usata e la chiave di sessione cifrata), seguite dal contenuto

cifrato con ChaCha20-Poly1305, un algoritmo moderno che garantisce sia riservatezza che integrità.

Nel contesto di questo progetto, *AGE*$_G$ è stato studiato come tecnologia di riferimento per la fase successiva del lavoro, che prevede la progettazione di *repository* cifrate con distribuzione dei permessi di accesso. In questa fase *AGE*$_G$ permette di cifrare il contenuto di una *repository* in modo che solo gli utenti autorizzati — identificati dalle loro chiavi pubbliche — possano decifrarla, senza dover condividere nessuna chiave segreta.


```
age-encryption.org/v1
-> ssh-ed25519 Yu405A MJl34qS7ezoxXy9FQjl5TVe1BLjrz1hYEpxVxaLqFy8
bs84Hjf93u1AyipJZ50oe2NxvEeZk1EHkY1/uDZW5pI
--- RxYyA9D3tPSdSdvlcGsVOp/0oOT9XMWCJ1ffjI7mEcs
‡□□□□Í,<.'›0¹êôx9□(NÎÿ
□P§Sø†UùË□Pµ[□=tî,…¼i'»@IÒþÙ#ëÇÜtlF„ô□¼x□□|J3]Fb“°Juž{§bŽ#□Àg□é¸{Œ□ÍT–□□·□F½9*ÃÒ
```


Esempio di un file cifrato con AGE

3.4 Sistemi di controllo versione

3.4.1 Storia e evoluzione

I sistemi di controllo versione (*Version Control System*, *VCS*$_G$) nascono dalla necessità pratica di tenere traccia delle modifiche al codice sorgente nel tempo. Prima della loro diffusione, era comune affidare il versionamento a convenzioni manuali: copie di file con date nel nome, cartelle numerate, archivi compressi. Questo approccio era fragile, difficile da gestire in team e privo di qualsiasi garanzia di integrità.

Il primo sistema formale di controllo versione ampiamente adottato fu **SCCS** (*Source Code Control System*), sviluppato da Bell Labs nel 1972. SCCS introduceva il concetto di *delta*: invece di salvare ogni versione completa del file, memorizzava solo le differenze rispetto alla versione precedente, riducendo significativamente lo spazio occupato.

RCS (*Revision Control System*), sviluppato nel 1982, migliorò SCCS rendendolo più accessibile e introducendo il concetto di *branching*, ossia la possibilità di sviluppare linee di codice parallele a partire da un punto comune.

Entrambi questi sistemi operavano però a livello di singolo file e su singola macchina. Il passo successivo fu l'introduzione dei sistemi *centralizzati*.

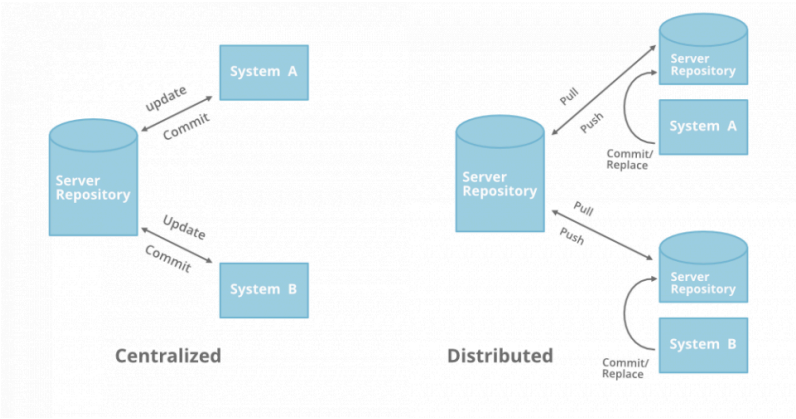
3.4.1.1 Sistemi centralizzati

CVS (*Concurrent Versions System*), negli anni "90, portò il controllo versione in rete. Per la prima volta più sviluppatori potevano lavorare contemporaneamente sullo stesso

progetto, con un server centrale che coordinava le modifiche. Il modello era semplice: il server custodisce l"intera storia del progetto; i client effettuano *checkout* (scaricano una versione) e *commit* (inviano le modifiche).

Subversion (SVN), rilasciato nel 2000, nacque esplicitamente come sostituto migliorato di CVS, correggendo molte delle sue limitazioni tecniche. SVN trattava l'intera struttura del progetto come un'unità atomica: un *commit*$_G$ poteva riguardare più file contemporaneamente, con la garanzia che o tutte le modifiche venivano salvate o nessuna.

Il limite fondamentale dei sistemi centralizzati era però strutturale: la presenza di un singolo punto di fallimento. Se il server non era raggiungibile, nessuno poteva fare *commit*$_G$. Se il server veniva perso, si perdeva l'intera storia del progetto.



Controllo versioni centralizzato vs distribuito

3.4.1.2 Sistemi distribuiti

Git$_G$, sviluppato da Linus Torvalds nel 2005 per gestire lo sviluppo del kernel Linux, rivoluzionò il campo introducendo un modello completamente distribuito. In *Git*$_G$ non esiste un server centrale: ogni sviluppatore possiede una copia completa dell'intera storia del progetto. I *commit*$_G$ avvengono localmente e possono essere sincronizzati con altri *repository*$_G$ in un secondo momento.

Questa architettura offre vantaggi significativi: si può lavorare offline, la storia del progetto è replicata su ogni macchina riducendo il rischio di perdita dei dati, e il *branching* è diventato un'operazione economica e centrale nel flusso di lavoro.

Mercurial, rilasciato nello stesso anno di *Git*$_G$, adottò un approccio simile ma con un'interfaccia considerata più accessibile. Oggi *Git*$_G$ domina il mercato, ma l'ecosistema dei *VCS*$_G$ distribuiti rimane vivo con strumenti come Fossil e Pijul.

3.4.2 Git in dettaglio

In *Git*$_G$ ogni oggetto — *blob* (contenuto di un file), *tree* (struttura di una directory), *commit*, *tag* — è identificato da un *hash*$_G$ SHA del suo contenuto. Questo significa che l'identità di ogni oggetto è determinata dal suo contenuto: due oggetti con lo stesso contenuto hanno lo stesso *hash*$_G$, e qualsiasi modifica produce un *hash*$_G$ diverso.

Un *commit* *Git*$_G$ contiene: il riferimento all'albero dei file in quello stato, il riferimento al *commit*$_G$ precedente (*parent*), i metadati dell'autore e del committer, il messaggio di *commit*$_G$. Questa struttura crea una catena crittograficamente collegata: modificare un *commit*$_G$ invalida tutti i *commit*$_G$ successivi, poiché i loro *hash*$_G$ dipendono dal *hash*$_G$ del precedente.

Git$_G$ supporta la firma crittografica dei *commit*$_G$ tramite GPG o *SSH*$_G$. Tuttavia questa funzionalità è opzionale e deve essere abilitata esplicitamente — non fa parte del flusso di lavoro standard.

3.4.3 RVC a confronto con Git

RVC$_G$ condivide con *Git*$_G$ il modello distribuito ma si differenzia in aspetti fondamentali di architettura e sicurezza.

Caratteristica	***Git*$_G$**	***RVC*$_G$**
Struttura	*Repository* con oggetti indicizzati	File ZIP navigabili su file-system
Identificazione *commit*$_G$	*Hash*$_G$ SHA dell'oggetto *commit*$_G$	Timestamp codificato in base36
Firma *commit*$_G$	Opzionale (GPG o *SSH*$_G$)	Integrata nell'architettura
Server centrale	Non richiesto ma comune	Non richiesto per design
Repository$_G$ multiple	Un remote alla volta tipicamente	Più *repository*$_G$ sincronizzate nativamente
Linguaggio	C	*CPL*$_G$

Confronto tra Git e RVC.

La differenza più significativa riguarda la sicurezza: mentre in *Git*$_G$ la firma è un'opzione che il singolo sviluppatore può scegliere di abilitare o meno, in *RVC*$_G$ è parte del modello

stesso. Ogni *commit* produce un file `.sig` che contiene gli *hash*$_G$ crittografici del contenuto e della catena precedente, costruendo una struttura analoga a una *blockchain*: modificare un *commit*$_G$ invalida tutti quelli successivi perché l'hash cumulativo non corrisponde più.

3.5 RVC: architettura e funzionamento

3.5.1 Struttura della repository

Una *repository* *RVC*$_G$ è una semplice cartella sul filesystem, senza strutture dati complesse o indici da mantenere. Ogni *commit* è rappresentato da due file:

- Un archivio **ZIP** contenente il *commit* del progetto nella versione corrispondente, incluso il file `.FileManifest` che descrive lo stato di tutti i file tracciati.
- Un file **.sig** contenente i metadati del *commit*$_G$ e, opzionalmente, la firma *SSH*$_G$.

I file seguono una convenzione di denominazione che codifica la struttura della storia:

```
1 <progetto>.<id>.<idPrecedente>.{autore}+tag.zip
2 <progetto>.<id>.sig
```

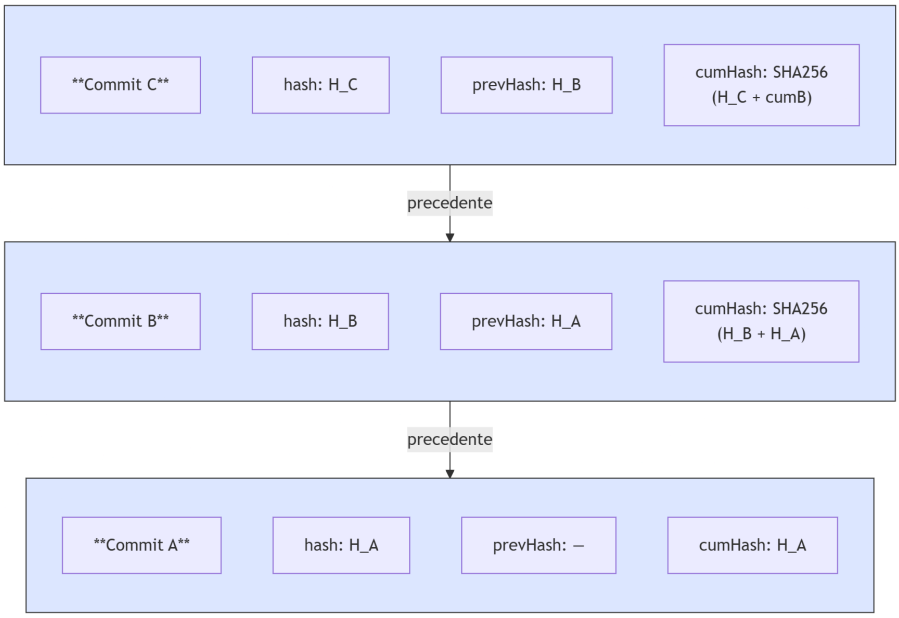
L'`id` è un timestamp codificato in base36 (cifre 0-9 e lettere A-Z), che permette l'ordinamento cronologico dei *commit*$_G$ semplicemente confrontando i nomi dei file. Il riferimento al *commit*$_G$ precedente è incorporato nel nome del file ZIP, rendendo la struttura della storia navigabile senza alcun indice aggiuntivo.

3.5.2 Il file .sig e la blockchain degli hash

Il file `.sig` è il cuore del sistema di sicurezza di *RVC*$_G$. Contiene in formato binario proprietario i seguenti campi:

- `author`: il nome dell'autore del *commit*$_G$
- `comment`: il messaggio del *commit*$_G$
- `fn`: il nome del file ZIP corrispondente
- `id`: l'identificativo del *commit*$_G$
- `prevId`: l'identificativo del *commit*$_G$ precedente
- `hash`: lo SHA256 del file ZIP di questo *commit*$_G$
- `prevHash`: lo SHA256 del file ZIP del *commit*$_G$ precedente
- `cumulativeHash`: lo SHA256 della concatenazione dell'hash attuale con il `cumulativeHash` del *commit*$_G$ precedente

Il `cumulativeHash` è la chiave della sicurezza: ogni *commit*$_G$ incorpora crittograficamente l'intera storia precedente. Verificare che il `cumulativeHash` di un *commit*$_G$ sia corretto significa verificare implicitamente che tutti i *commit*$_G$ precedenti siano integri.



Struttura del file .sig e la catena degli hash cumulativi

Dopo i metadati, il file `.sig` contiene una firma *SSH*$_G$ nel formato standard:

```
1 -----BEGIN SSH SIGNATURE-----
2 ...
3 -----END SSH SIGNATURE-----
```

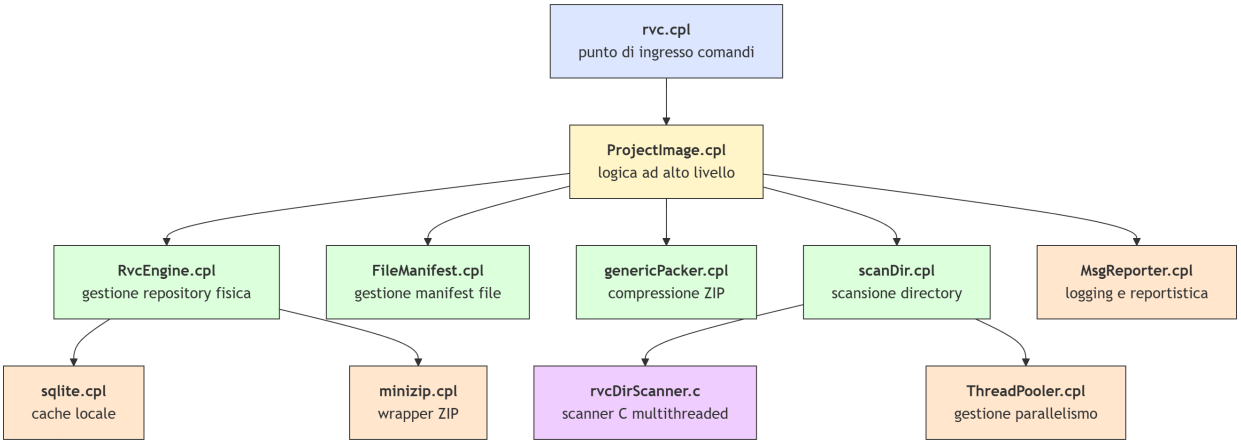
Questa firma attesta che l'autore dichiarato ha effettivamente prodotto il *commit*$_G$, rendendo ogni modifica crittograficamente attribuibile.

3.5.3 Il linguaggio CPL

RVC$_G$ è scritto in ***CPL***$_G$ (*CodePainter Language*), un linguaggio proprietario sviluppato da Zucchetti S.p.A. *CPL*$_G$ è un linguaggio interpretato tipizzato staticamente, con supporto a classi, moduli e gestione dei file. Viene eseguito tramite un interprete (`cpl.exe`) che supporta sia interpretazione diretta che compilazione *JIT*$_G$.

Le caratteristiche principali che distinguono *CPL*$_G$ dai linguaggi comuni includono la sintassi di assegnazione con `:=`, l'assenza dell'istruzione `return` esplicita (si usa invece la variabile implicita `result`), la dichiarazione obbligatoria di tutte le variabili in cima alla funzione prima di qualsiasi blocco di codice, e la distinzione tra `func` (funzione con valore di ritorno) e `proc` (procedura senza valore di ritorno).

Il codice sorgente di *RVC*$_G$ è organizzato in diversi moduli *CPL*$_G$, ciascuno con responsabilità ben definite: `ProjectImage.cpl` contiene la logica ad alto livello, `RvcEngine.cpl` gestisce la *repository*$_G$ fisica, `FileManifest.cpl` gestisce il *manifest*$_G$ dei file tracciati.



Struttura dei moduli principali di RVC

3.5.4 Flusso di un commit

Quando un utente esegue `rvc commit`, il sistema esegue i seguenti passi:

1. Legge il file `.FileManifest` nella directory di lavoro per conoscere lo stato corrente del progetto.
2. Scansiona la directory e calcola le differenze rispetto allo stato precedente.
3. Crea un archivio ZIP con i file modificati e il nuovo `.FileManifest`.
4. Calcola lo SHA256 dell'archivio ZIP.
5. Recupera l'hash e il `cumulativeHash` del *commit*$_G$ precedente dal suo file `.sig`.
6. Calcola il nuovo `cumulativeHash` come SHA256 della concatenazione dell'hash attuale con il `cumulativeHash` precedente.
7. Crea il file `.sig` con tutti i metadati.
8. Esegue `ssh-keygen -Y sign` per firmare il file `.sig` e accoda la firma al file.
9. Copia i due file nella *repository*$_G$.

Capitolo 4

Requisiti e modello di sicurezza

In questo capitolo viene definito il modello di sicurezza che un sistema di versionamento distribuito moderno dovrebbe soddisfare. Partendo dalle proprietà fondamentali richieste, vengono analizzate le scelte architetturali, i trade-off e le motivazioni che portano alla definizione di un sistema gerarchico di fiducia, di livelli di sicurezza configurabili e di requisiti formali. Il capitolo si conclude con un analisi del divario che confronta il modello ideale con lo stato attuale di RVC, identificando le aree di intervento affrontate nei capitoli successivi.

4.1 Sicurezza strutturale nei sistemi di versionamento distribuito

Un sistema di versionamento distribuito è considerato sicuro quando garantisce, per ogni operazione e indipendentemente dal canale di distribuzione, le seguenti proprietà fondamentali.

- **Integrità** — il contenuto di ogni *commit*$_G$ non può essere alterato senza che la modifica sia rilevabile. Questa proprietà è garantita da funzioni di *hash*$_G$ crittografiche: dato un archivio ZIP, la sua impronta *SHA-256*$_G$ è univoca e deterministica. Qualsiasi modifica — anche di un singolo byte — produce un hash completamente diverso. Un sistema che garantisce l'integrità permette a chiunque di verificare che il contenuto ricevuto sia identico a quello prodotto dall'autore, senza dover contattare nessuna fonte autoritativa.
- **Autenticità** — ogni *commit*$_G$ è crittograficamente attribuibile all'autore che l'ha prodotta. L'autenticità è garantita dalla *firma digitale*$_G$: l'autore firma il *commit*$_G$ con la propria *chiave privata*$_G$ e chiunque può verificare la firma usando la corrispondente *chiave pubblica*$_G$. Senza autenticità, un sistema può garantire che i dati non siano stati modificati durante il trasferimento, ma non può garantire chi li abbia prodotti originariamente.
- **Non ripudio** — un autore non può negare di aver prodotto una *commit*$_G$ firmata con la propria *chiave privata*$_G$. Il non ripudio è una conseguenza diretta della firma digitale: poiché solo il possessore della chiave privata può produrre una firma valida, la presenza di una firma valida è prova crittografica della paternità. Questa proprietà

è rilevante in contesti contrattuali e legali — se un fornitore di software distribuisce codice firmato, non può successivamente sostenere di non averlo prodotto.

- **Ordine verificabile** — la sequenza temporale delle modifiche è verificabile e non manipolabile retroattivamente. Questa proprietà è la più complessa da garantire in un sistema distribuito. Non è sufficiente che ogni *commit*$_G$ sia integra e autentica — è necessario anche che la loro sequenza sia crittograficamente vincolata, in modo che inserire, rimuovere o riordinare commit sia rilevabile. Questa proprietà è garantita dalla struttura a catena degli hash: ogni *commit*$_G$ incorpora l'hash della precedente, rendendo impossibile modificare un elemento della catena senza invalidare tutti quelli successivi.

4.1.1 Confronto con i sistemi esistenti

In *Git*$_G$ l'integrità è garantita dalla catena di hash *SHA-256*$_G$ — modificare una *commit*$_G$ invalida tutte quelle successive. Autenticità e non ripudio sono invece opzionali: la firma crittografica tramite *SSH*$_G$ o GPG deve essere abilitata esplicitamente e non fa parte del flusso di lavoro standard. L'ordine temporale non è verificabile in senso assoluto — *Git*$_G$ non verifica i timestamp e permette la creazione di *commit*$_G$ con date arbitrarie:

```bash
GIT_AUTHOR_DATE="2020-01-01T00:00:00" git commit -m "commit retrodatata"
```

Il risultato è una *commit*$_G$ inserita nella storia con una data arbitraria, indistinguibile da una *commit*$_G$ legittima. Questa non è una vulnerabilità ma una scelta progettuale documentata — *Git*$_G$ garantisce l'ordine relativo tramite la catena di hash, non l'ordine temporale assoluto.

La distinzione tra disponibilità e correttezza di un sistema centralizzato è emersa concretamente il 23 aprile 2026, quando un bug nella funzionalità di merge queue di GitHub ha causato la generazione silenziosa di *commit*$_G$ errate per 2.092 pull request in 658 *repository*$_G$. Le modifiche precedentemente unite sono state ripristinate dai merge successivi senza alcun avviso. La piattaforma era tecnicamente operativa durante tutto l'incidente — gli sviluppatori potevano ancora fare push, aprire pull request e fare merge. Il fatto che il merge stesse corrompendo silenziosamente il codice non risultava come incidente nella dashboard di stato. Disponibilità e correttezza non sono la stessa proprietà: un sistema può essere raggiungibile e simultaneamente produrre risultati errati. Un sistema distribuito che incorpora le garanzie crittografiche descritte sopra non dipende dalla correttezza del server per verificare l'integrità dei propri dati.

4.1.2 Applicazione in RVC

RVC$_G$ garantisce l'integrità tramite il `cumulativeHash` — ogni *commit*$_G$ incorpora crittograficamente l'intera storia precedente. La firma *SSH*$_G$ garantisce autenticità e non ripudio. L'ordine temporale presenta la stessa limitazione di *Git*$_G$ — gli identificativi sono timestamp codificati in base36 basati sull'orologio della macchina. Il `cumulativeHash` garantisce però l'ordine **relativo**: anche con timestamp errati o manipolati, la sequenza crittografica è verificabile e non manipolabile senza invalidare l'intera catena. Questa limitazione viene documentata come scelta consapevole — per il contesto d'uso di *RVC*$_G$ la garanzia di ordine relativo è sufficiente e la complessità aggiuntiva di un sistema di timestamping certificato non è giustificata.

Per mitigare parzialmente questa limitazione, il modello proposto prevede che l'identificativo di ogni *commit*$_G$ sia composto da due componenti: il timestamp codificato in base36 per mantenere l'ordinamento cronologico visivo, e una porzione dell'hash del contenuto per garantire l'unicità crittografica. Un identificativo nella forma `0Q6JTD7XVZ_A3F2B1C4` — dove la prima parte è il timestamp e la seconda sono i primi otto caratteri dell'*hash*$_G$ *SHA-256*$_G$ dell'archivio ZIP — rende praticamente impossibile la collisione intenzionale mantenendo la leggibilità e l'ordinamento per nome file. Questa modifica è identificata come requisito nel modello ideale e discussa nella gap analysis nella Sezione 4.3.

4.2 Requisiti di sicurezza formali

A partire dalle proprietà definite nella sezione precedente, è possibile derivare un insieme di requisiti formali che un sistema di versionamento distribuito sicuro deve soddisfare. Ogni requisito è classificato per priorità — obbligatorio (O), desiderabile (D) o facoltativo (F) — e verrà ripreso nella *gap analysis* per valutare lo stato attuale di *RVC*$_G$.

Codice	Descrizione	Priorità
RS01	Ogni *commit*$_G$ deve essere verificabile tramite *hash*$_G$ crittografico del proprio contenuto	O
RS02	La catena degli hash deve essere verificabile in modo che qualsiasi modifica a una *commit*$_G$ invalidi tutte le successive	O
RS03	La prima *commit*$_G$ di ogni *repository*$_G$ deve costituire una radice di fiducia verificabile autonomamente	O
RS04	Il sistema deve supportare e imporre la firma digitale tramite chiave SSH per ogni commit nei progetti configurati con livello di sicurezza maggiore o uguale a 1	O
RS05	Il sistema deve supportare una gerarchia di fiducia a tre livelli: amministratore, responsabile e dipendente	O
RS06	I permessi di scrittura devono essere configurabili per progetto tramite un file di autorizzazione versionato	O
RS07	La revoca di un'identità deve essere efficace dalla *commit*$_G$ successiva alla modifica del file di autorizzazione	O
RS08	La successione di un responsabile deve essere gestita esclusivamente dall'amministratore del sistema	D
RS09	Gli identificativi dei *commit*$_G$ devono essere univoci e non manipolabili tramite timestamp arbitrari	O
RS10	Il sistema deve documentare esplicitamente le proprie limitazioni in termini di ordine temporale assoluto	O
RS11	Il sistema deve supportare livelli di sicurezza configurabili per progetto, non abbassabili nel tempo	D
RS12	Il contenuto delle *commit*$_G$ deve poter essere cifrato con *AGE*$_G$ per progetti riservati	D
RS13	La gestione dei destinatari nei progetti cifrati deve supportare modalità configurabili di visibilità	D
RS14	I branch compromessi devono poter essere chiusi con una *commit*$_G$ firmata che ne attesti la compromissione	D

Requisiti di sicurezza del modello proposto.

I requisiti obbligatori definiscono le proprietà minime senza le quali il sistema non può essere considerato sicuro per il contesto d'uso descritto. I requisiti desiderabili estendono il modello con funzionalità che aumentano significativamente il livello di sicurezza, ma la cui assenza non compromette le garanzie fondamentali.

I requisiti RS01, RS02 e RS12 corrispondono alla proprietà di integrità e alla definizione della radice di fiducia. RS03 garantisce le proprietà di autenticità e non ripudio. RS04, RS05, RS06 e RS13 definiscono il modello di gestione delle identità e dei permessi. RS07 e RS11 affrontano il problema dell'ordine temporale. Infine, i requisiti RS08, RS09, RS10 e RS14 estendono il modello con funzionalità avanzate, quali la sicurezza configurabile, la cifratura e la risposta alle compromissioni.

4.3 Gap analysis

Capitolo 5

Simulazione scenari di attacco

In questo capitolo effettuo l'analisi degli utenti, sviluppo le user stories e compongo la lista dei requisiti dividendoli per tipologia e necessità.

Capitolo 6

Miglioramenti implementati

In questo capitolo effettuo l'analisi degli utenti, sviluppo le user stories e compongo la lista dei requisiti dividendoli per tipologia e necessità.

Capitolo 7

Conclusioni

In questo capitolo traggo le conclusioni sul progetto.

7.1 Consuntivo finale

Una volta terminato il progetto ho redatto il consuntivo orario finale nella 7.1 che suddivide in maniera approssimata le ore dedicate alle varie fasi.

Fase	**Ore**
Onboarding del progetto	5
Analisi dei requisiti	30
...	...
Totale	320

Consuntivo orario finale.

7.2 Raggiungimento degli obiettivi

7.3 Requisiti soddisfatti

Arrivato alla fine del progetto ho implementato...

Tipo	Mandatory	Desirable	Optional	Somma
Functional	0/2	0/0	0/0	0/2
Qualitative	0/1	0/1	0/1	0/3
Constraint	0/1	0/0	0/0	0/1
Totale	**0/4**	**0/1**	**0/1**	**0/6**

Riepilogo dei requisiti soddisfatti.

7.4 Rischi occorsi e mitigati

I rischi emersi durante lo stage sono riportati in 7.3.

Descrizione	Mitigazione
R1 – Descrizione del rischio	Soluzione

Rischi occorsi con la loro mitigazione.

7.5 Valutazione personale

Glossario

AGE – Actually Good Encryption: Strumento moderno per la cifratura di file. Utilizza algoritmi crittografici contemporanei come X25519 e ChaCha20-Poly1305, con un'interfaccia volutamente semplice. 3., 4., 8., 9., 11., 14., 15., 24.

Black-box: Approccio di analisi della sicurezza condotto senza accesso al codice sorgente, osservando esclusivamente il comportamento esterno del sistema in risposta agli input forniti. 5.

Chiave privata: Componente segreta di una coppia di chiavi asimmetriche. Deve essere custodita esclusivamente dal proprietario e viene utilizzata per decifrare messaggi o produrre firme digitali. 21.

Chiave pubblica: Componente pubblica di una coppia di chiavi asimmetriche. Può essere distribuita liberamente e viene utilizzata per cifrare messaggi o verificare firme digitali. 21.

Commit: Unità atomica di modifica in un sistema di versionamento. Rappresenta uno snapshot dello stato dei file in un determinato momento, accompagnato da metadati quali autore, data e messaggio descrittivo. v, 4., 16., 17., 18., 19., 20., 21., 22., 23., 24.

CPL – CodePainter Language: Linguaggio di programmazione proprietario sviluppato da Zucchetti S.p.A. Interpretato, tipizzato staticamente, con sintassi simile al Pascal. Utilizzato per lo sviluppo di RVC e dei prodotti software Zucchetti. v, 4., 5., 8., 9., 17., 19.

Ed25519: Algoritmo di firma digitale basato sulla curva ellittica Curve25519. Produce chiavi di 256 bit con un livello di sicurezza equivalente a RSA con chiavi da 3000 bit, risultando più efficiente e compatto. 8., 13., 14.

Firma digitale: Meccanismo crittografico che garantisce autenticità e integrità di un documento. Prodotta con la chiave privata del firmatario, può essere verificata da chiunque possieda la chiave pubblica corrispondente. 21.

Git: Sistema di versionamento distribuito open source, creato da Linus Torvalds nel 2005. Garantisce l'integrità della storia tramite una catena di hash crittografici. v, 2., 16., 17., 22., 23.

Hash: Funzione matematica che trasforma un dato di dimensione arbitraria in una stringa di lunghezza fissa. Una minima modifica dell'input produce un output completamente diverso, garantendo l'integrità dei dati. v, vi, 2., 9., 12., 17., 18., 21., 23., 24.

JIT – Just-In-Time: Tecnica di compilazione che traduce il codice intermedio in codice macchina durante l'esecuzione, migliorando le prestazioni rispetto alla sola interpretazione. 19.

Manifest: Nel contesto di RVC, file che descrive lo stato di tutti i file tracciati in un progetto in un dato momento: nomi, dimensioni, hash e versione di ogni file. 19.

Namespace: Nel contesto della firma SSH, stringa che identifica il contesto d'uso di una firma e previene attacchi di riutilizzo: una firma prodotta con un certo namespace non può essere usata in un contesto diverso. 14.

Passphrase: Sequenza di parole o caratteri utilizzata per proteggere una chiave privata. Più lunga e complessa di una password tradizionale, viene richiesta ogni volta che si utilizza la chiave. 14.

Repository: Archivio che contiene la storia completa delle modifiche di un progetto sotto controllo versione, inclusi tutti i commit, i branch e i tag. v, 8., 9., 16., 17., 19., 20., 22., 24.

RVC – Repositoryless Version Control: Sistema di versionamento distribuito sviluppato da Zucchetti S.p.A. come alternativa a Git. Garantisce l'integrità dei contenuti tramite verifica crittografica degli hash di ogni commit, permettendo a qualsiasi utente di accertare autonomamente l'autenticità della repository ricevuta. v, vi, ix, 2., 3., 4., 5., 8., 9., 11., 14., 17., 18., 19., 23.

SHA-256 – Secure Hash Algorithm 256: Funzione di hash crittografica che produce un'impronta digitale di 256 bit. Ampiamente utilizzata per verificare l'integrità dei dati grazie alla sua resistenza alle collisioni e alla non invertibilità. 21., 22., 23.

SSH – Secure Shell: Protocollo crittografico per la comunicazione sicura su reti non affidabili. Utilizza crittografia asimmetrica per l'autenticazione e la cifratura del canale. v, vi, 2., 3., 4., 8., 9., 11., 13., 14., 17., 18., 19., 22., 23.

VCS – Version Control System: Sistema software che registra le modifiche ai file nel tempo, permettendo di recuperare versioni precedenti, confrontare cambiamenti e coordinare il lavoro di più sviluppatori. 15., 16.

White-box: Approccio di analisi della sicurezza con piena visibilità del codice sorgente e dell'architettura interna del sistema. Si contrappone all'analisi black-box, condotta senza accesso ai sorgenti. 5.